Esercizi — variabili, istruzioni, espressioni

Lezione coperta: `L04` — variabili, istruzioni, espressioni, logica booleana

La selezione non è qui. `if`, `elif` e `else` hanno una scheda loro, `L05_esercizi.md`. Qui non ne serve nessuno: tutto si fa con assegnazioni, operatori ed espressioni.

Come si lavora. Gli esercizi salgono in tre gradini: prima **seguire** un codice già scritto e dire che cosa produce, poi **trovare** l'errore in uno che quasi funziona, infine **scriverne** uno tu. Le soluzioni sono sotto ogni esercizio: aprile dopo aver provato.

Il primo gradino si fa senza calcolatore, con carta e penna: è il modo in cui si impara a prevedere che cosa farà un programma invece di scoprirlo. Dal secondo in poi conviene avere il terminale aperto.

Gradino 1 — seguire il codice

Esercizio 1 — Tenere traccia (difficoltà: •)

Senza eseguirlo, di' quanto valgono `a`, `b` e `c` alla fine, e che cosa stampa l'ultima riga.

```
a = 3
b = a + 10
c = a + b
c = 4 * c
print(f"questo e' un computer a {c} bit")
```

Soluzione

Mostra la soluzione

`a` vale **3**, `b` vale **13**, `c` vale **64**, e stampa

```
questo e' un computer a 64 bit
```

Il passaggio dove si sbaglia è il quarto: `c = 4 * c` non è un'equazione, è un'istruzione. Si calcola prima la destra — `4 * 16`, cioè 64 — e poi si associa il risultato al nome `c`. Il vecchio valore di `c` serve a calcolare il nuovo e poi non esiste più.

Se hai scritto i valori dopo ogni riga, hai fatto la cosa giusta.

Esercizio 2 — Le due divisioni (difficoltà: •)

Quanto vale ciascuna di queste espressioni, e di che tipo è il risultato?

```
17 / 5
17 // 5
17 % 5
17.0 // 5
```

Soluzione

Mostra la soluzione

espressione	vale	tipo
`17 / 5`	`3.4`	float
`17 // 5`	`3`	int
`17 % 5`	`2`	int
`17.0 // 5`	`3.0`	float

Due cose da portarsi dietro. `/` restituisce **sempre** un float, anche quando la divisione è esatta: `10 / 5` vale `2.0`, non `2`. E `//` tronca ma **conserva il tipo** di quello che riceve: fra interi dà un intero, con un float dà un float.

Esercizio 3 — Il + non somma sempre (difficoltà: •)

Che cosa produce ciascuna riga?

```
"5" + "3"
5 + 3
3 * "ab"
"5" + 3
```

Soluzione

Mostra la soluzione

```
"5" + "3"   ->  '53'
5 + 3       ->  8
3 * "ab"    ->  'abab'
"5" + 3     ->  TypeError: can only concatenate str (not "int") to str
```

`+` e `*` fanno cose diverse a seconda di che cosa ricevono: con i numeri sommano e moltiplicano, con le stringhe concatenano e ripetono.

La riga pericolosa è la prima, non l'ultima. L'ultima si ferma con un errore chiaro; la prima produce `'53'` e prosegue. Se quei due valori venissero da `input()`, che restituisce sempre stringhe, avresti un programma che somma sbagliato senza dirtelo.

Esercizio 4 — Una copia che non è una copia (difficoltà: •)

Che cosa stampa?

```
a = 7
b = a
a = 99
print(b)
```

Soluzione

Mostra la soluzione

Stampa **7**.

`b = a` non crea un legame fra i due nomi: dà a `b` lo **stesso oggetto** che ha `a` in quel momento. Quando poi `a = 99` associa ad `a` un oggetto **nuovo**, `b` resta dov'era.

Non è «`b` ha copiato il valore»: è che l'assegnazione a `a` ha spostato solo `a`. La differenza conta quando arriveremo alle liste, dove lo stesso oggetto può essere modificato invece che sostituito.

Gradino 2 — trovare l'errore

Esercizio 5 — Lo script rotto (difficoltà: ••)

Questo programma dovrebbe stampare l'area di un quadrato e quella di un cerchio. Ha **cinque** errori. Trovali tutti, poi correggilo e provalo.

```
lato = 5
raggio = 10
PI = 3.74
print("Un quadrato di lato: " + lato + " ha un'area di: " lato**2)
area = (PI * raggio) ** 2
print("Un cerchio di raggio: " + raggio + " ha un'area di: area")
```

Suggerimento: **tre errori fermano il programma, due no.** Quelli che non lo fermano sono i più importanti.

Soluzione

Mostra la soluzione

I tre che fermano il programma, nell'ordine in cui li incontri:

1. riga 4, manca un `+` fra `" ha un'area di: "` e `lato**2`. Python dice `SyntaxError: invalid syntax. Perhaps you forgot a comma?` — il suggerimento è sbagliato, ma il punto indicato è giusto;
2. riga 4, `"..." + lato` concatena una stringa e un intero: `TypeError: can only concatenate str (not "int") to str`. Serve `str(lato)`, oppure una f-string;
3. riga 6, stesso errore con `raggio`.

I due che non lo fermano, e sono quelli veri:

4. `PI = 3.74`. Il valore giusto è `3.14159...`: così com'è sbaglia del **19%**, e nessuno protesta;
5. `area = (PI * raggio) ** 2` **non è l'area del cerchio**. La formula è `PI * raggio**2`, cioè `314.159`; quella scritta dà `1398.76`, quattro volte e mezzo tanto. È un'espressione legittima, quindi Python la calcola e prosegue.

Un sesto problema, che non è un errore ma un difetto: la riga 6 stampa la **parola** `area` invece del valore, perché è dentro le virgolette.

Una versione corretta:

```
lato = 5
raggio = 10
PI = 3.14159
print(f"Un quadrato di lato {lato} ha un'area di {lato**2}")
area = PI * raggio**2
print(f"Un cerchio di raggio {raggio} ha un'area di {area:.2f}")
```

La morale. Gli errori che si fermano te li segnala Python; quelli che non si fermano li devi trovare tu, e sono quelli che finiscono nei risultati.

Esercizio 6 — Quello che entra è testo (difficoltà: ••)

L'utente scrive `5` e poi `3`. Che cosa stampa questo programma, e perché non è quello che ci si aspetta?

```
a = input("primo numero: ")
b = input("secondo numero: ")
print(a + b)
```

Poi correggilo.

Soluzione

Mostra la soluzione

Stampa **53**, non 8.

`input()` restituisce **sempre una stringa**, anche quando l'utente batte delle cifre. Su due stringhe `+` concatena.

La correzione è convertire **appena si legge**:

```
a = int(input("primo numero: "))
b = int(input("secondo numero: "))
print(a + b)
```

Ogni riga che arriva da una tastiera, da un file o da un CSV è testo finché non dici tu il contrario. È l'errore più frequente delle prime settimane.

Esercizio 7 — In che ordine (difficoltà: ••)

Prevedi il risultato di ciascuna, poi verifica.

```
2 + 3 * 4
10 - 3 - 2
100 / 5 / 2
-2 ** 2
2 ** 3 ** 2
```

Soluzione

Mostra la soluzione

```
2 + 3 * 4    ->  14      la moltiplicazione viene prima
10 - 3 - 2   ->  5       da sinistra: (10-3)-2
100 / 5 / 2  ->  10.0    da sinistra: (100/5)/2
-2 ** 2      ->  -4      la potenza batte il meno: -(2**2)
2 ** 3 ** 2  ->  512     da DESTRA: 2**(3**2) = 2**9
```

Le prime tre si spiegano con due regole: la moltiplicazione e la divisione vengono prima della somma, e a parità di priorità si va da sinistra a destra.

Le ultime due no, e non sono memorabili. La potenza è **l'unico operatore che associa a destra**, e il meno unario perde contro di lei. Nessuno deve ricordarsele: si scrivono le parentesi.

Esercizio 8 — Nomi che non si possono usare (difficoltà: ••)

Quali di questi sono nomi di variabile validi in Python 3? Per quelli che non lo sono, di' perché.

```
area        2lati       lato_1      lato-1
Area        class       print       _temp
città       my var      True        media.totale
```

Soluzione

Mostra la soluzione

Validi: `area`, `lato_1`, `Area`, `print`, `_temp`, `città`.

Non validi:

nome	perché
`2lati`	non può **cominciare** con una cifra
`lato-1`	il trattino è l'operatore di sottrazione
`class`	è una **parola riservata**
`my var`	contiene uno spazio
`True`	è una parola riservata, dalla versione 3
`media.totale`	**non è un nome**: il punto significa «l'attributo `totale` dell'oggetto `media`»

Due casi che sorprendono.

`print` è valido, perché in Python 3 non è una parola chiave ma una funzione: puoi scrivere `print = 5`. Funziona, e da quel momento `print("ciao")` dà `TypeError: 'int' object is not callable`. Il linguaggio te lo lascia fare, quindi non farlo.

`città` è valido: Python 3 accetta le lettere accentate nei nomi. Resta una cattiva idea — chi legge il codice su un'altra tastiera fatica — ma non è un errore.

E un terzo, se l'hai provato: **`media.totale` non dà `SyntaxError`**. Il punto è un'operazione legittima, quindi Python prova a eseguirla e si ferma dopo, con `NameError: name 'media' is not defined` oppure, se `media` esiste, `AttributeError`. Non è un nome rifiutato: è un nome che Python non sta nemmeno leggendo come tale.

Gradino 3 — scrivere

Esercizio 9 — Lo scambio (difficoltà: •••)

`a` vale 3 e `b` vale 8. Scrivi le istruzioni che fanno diventare `a` uguale a 8 e `b` uguale a 3.

Poi prova questa soluzione e spiega che cosa non va:

```
a = b
b = a
```

Soluzione

Mostra la soluzione

La soluzione proposta dà **8 e 8**: la prima riga sovrascrive `a`, e con essa l'unica copia del 3, che a quel punto non è più da nessuna parte.

Due modi corretti. Con una variabile d'appoggio, che funziona in qualunque linguaggio:

```
tmp = a
a = b
b = tmp
```

Oppure alla maniera di Python, dove la destra si calcola **tutta prima** e poi si assegna:

```
a, b = b, a
```

È la stessa cosa dei bicchieri: per scambiare due liquidi serve un terzo bicchiere, oppure due mani.

Esercizio 10 — La media, scritta bene (difficoltà: •••)

Le massime di tre giorni sono 5, 6 e 8 gradi. Scrivi un programma che calcoli la media e la stampi dentro una frase, con **due cifre decimali**.

Soluzione

Mostra la soluzione

```
t1 = 5
t2 = 6
t3 = 8
media = (t1 + t2 + t3) / 3
print(f"La media delle tre massime è {media:.2f} gradi")
```

```
La media delle tre massime è 6.33 gradi
```

Senza `:.2f` la f-string stamperebbe `6.333333333333333`, che è esatto e illeggibile.

Due errori frequenti: scrivere `t1 + t2 + t3 / 3`, che divide solo l'ultimo — servono le parentesi — e dividere per un numero scritto a mano quando i valori sono tanti, invece di contarli.

Esercizio 11 — Dalla frase all'espressione (difficoltà: •••)

Scrivi l'espressione booleana che vale `True` quando la frase è vera. Non serve nessun `if`: solo l'espressione.

1. «la temperatura è sopra 30 **e** l'umidità sopra 70»

2. «la temperatura **non** è compresa fra 18 e 24»
3. «il giorno **non** è afoso», dove afoso è la condizione 1

Per la terza, scrivila **senza usare** `not` davanti a tutto.

Soluzione

Mostra la soluzione

```
temperatura > 30 and umidita > 70
not (18 <= temperatura <= 24)
temperatura <= 30 or umidita <= 70
```

La seconda si può scrivere anche `temperatura < 18 or temperatura > 24`: sono equivalenti. La forma `18 <= temperatura <= 24` è quella che Python permette e quasi nessun altro linguaggio.

La terza è **De Morgan**: negando un `and` i confronti si negano uno per uno **e l'operatore si ribalta** in `or`. L'errore classico è lasciare `and`:

```
temperatura <= 30 and umidita <= 70     # SBAGLIATA
```

Quella dice «non fa caldo **e** non è umido», cioè che devono saltare entrambe le condizioni. «Non afoso» invece vuol dire che ne basta **una**.

Esercizio 12 — L'anno bisestile (difficoltà: •••)

Un anno è bisestile se è divisibile per 4, **tranne** gli anni divisibili per 100, **ma** quelli divisibili per 400 lo sono comunque.

Scrivi l'espressione booleana che lo dice, e verificala su 2024, 1900, 2000 e 2026.

Soluzione

Mostra la soluzione

```
bisestile = (anno % 4 == 0 and anno % 100 != 0) or anno % 400 == 0
```

anno		perché
2024	`True`	divisibile per 4, non per 100
1900	`False`	divisibile per 100 ma non per 400
2000	`True`	divisibile per 400
2026	`False`	non divisibile per 4

Il nodo è la struttura, non l'aritmetica: le parentesi attorno al primo gruppo **servono**. Senza, `and` lega più stretto di `or` e l'espressione resta la stessa per caso — ma scriverle rende leggibile la regola, che è fatta di una condizione generale con un'eccezione e un'eccezione all'eccezione.

`%` qui serve a chiedere «è divisibile?»: un numero è divisibile per `n` quando il resto della divisione per `n` è zero.